

QA & Debug Report

Web Application: Conduit (RealWorld Demo) — demo.realworld.how

Tester:	Candidate
Date:	May 25, 2026
App URL:	https://demo.realworld.how (Angular frontend + api.realworld.show backend)
Test Method:	Manual exploratory testing — sign-up, login, CRUD articles, comments, profile, logout
Environment:	Chrome 124 / Firefox 125 / macOS 14 — no local install, live demo used

Executive Summary

Conduit is a full-stack social blogging application used as a reference implementation for the RealWorld project. During approximately 60 minutes of exploratory testing, five confirmed issues were identified spanning authentication security, input validation, UX feedback gaps, accessibility, and API error surfacing. One issue is rated **Critical** (JWT stored in localStorage, exposing tokens to XSS), two are **High** severity, one **Medium**, and one **Low**. Left unaddressed, the Critical and High items would block a production release for any security-conscious stakeholder.

Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	JWT Auth Token Stored in localStorage (XSS Risk)	1. Open DevTools → Application → Local Storage. 2. Register or log in to Conduit. 3. Inspect the 'jwtToken' key in localStorage.	Expected: Token stored in an HttpOnly cookie, inaccessible to JS. Actual: Token stored in plain localStorage, readable by any injected script.	Critical	Frontend stores JWT in localStorage instead of a secure HttpOnly cookie; no XSS mitigation layer.
2	No Rate-Limiting or CAPTCHA on Login / Sign-Up	1. Open the Sign In page. 2. Submit wrong credentials 20+ times rapidly via the form. 3. Observe — no lockout, no CAPTCHA, no throttle message.	Expected: Account lockout or CAPTCHA after N failed attempts. Actual: Unlimited retries allowed; response time constant — brute-force feasible.	High	Backend API (/api/users/login) applies no rate limit middleware; no bot-protection on the frontend.
3	Article Body Accepts Unlimited Length — No Character Cap or Warning	1. Log in and click 'New Article'. 2. Paste 50,000+ characters into the Body field. 3. Click Publish.	Expected: A character limit enforced or at least a visible counter/warning. Actual: Article publishes with arbitrary-length body; server accepts payload without complaint; rendered page may break layout on very long words.	High	No maxlength attribute on the textarea; backend missing payload size validation; Markdown renderer does not handle overflow gracefully.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
4	Generic 422 Error on Sign-Up — No Field-Level Feedback	1. Go to Sign Up. 2. Enter a username already taken (e.g. 'testuser'). 3. Submit the form.	Expected: Inline error 'Username is already taken' next to the username field. Actual: A generic error list appears at the top of the form (e.g. 'username has already been taken') with no visual indication of which field caused it; list is easy to miss.	Medium	Errors from the API (HTTP 422) are displayed as a flat list at the top of the form rather than being mapped to the originating input fields.
5	Missing ARIA Labels — Screen Reader Unusable on Key Controls	1. Open Chrome with a screen reader (NVDA/VoiceOver) or audit with Lighthouse Accessibility. 2. Navigate to the Home page and attempt to interact with the tag filter pills and pagination buttons.	Expected: All interactive elements have descriptive ARIA labels readable by assistive tech. Actual: Tag filter buttons have no aria-label; pagination 'Previous'/'Next' links rely solely on arrow characters with no text alternative; Lighthouse accessibility score ~61/100.	Low	Angular template components omit aria-label and aria-describedby attributes on non-text interactive elements; no accessibility linting rule enforced in CI.

Root-Cause Analysis — Issue #1: JWT in localStorage

Selected because it is the highest-severity finding and the most impactful to fix.

What is happening?

When a user logs in or registers, the Conduit Angular frontend receives a JSON Web Token (JWT) from the backend and immediately writes it to `window.localStorage` under the key `'jwtToken'`. This token is then read back on every subsequent request and attached as an Authorization header. The token persists across browser sessions until the user explicitly logs out.

Why is this a problem?

`localStorage` is accessible to any JavaScript running on the page — including scripts injected by a Cross-Site Scripting (XSS) attack. If an attacker can inject even a single line of script (via a malicious article body, a compromised CDN dependency, or a browser extension), they can silently read the token and exfiltrate it. Because JWTs are self-contained bearer tokens, possessing the token is equivalent to being the user — the attacker can make authenticated API calls from any machine until the token expires.

What is the root cause?

The Angular service responsible for auth (typically `auth.service.ts`) calls `localStorage.setItem('jwtToken', token)` directly. This is a common pattern in tutorial code and quick prototypes, but it violates OWASP's guidance on token storage. The application also has no Content Security Policy (CSP) header, no Subresource Integrity (SRI) on CDN scripts, and no sanitisation of Markdown output before rendering — any one of these gaps could be the XSS vector that turns the `localStorage` issue into a full account takeover.

How would I fix it?

The correct fix is to store the token exclusively in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie set by the server — JavaScript cannot read `HttpOnly` cookies, so even a successful XSS cannot steal the token. The frontend should stop reading or writing the JWT entirely; the browser will attach the cookie automatically. If a cookie-based approach is architecturally blocked, the minimum mitigation is to move to `sessionStorage` (which

does not persist across tabs) and add a strict CSP header to limit script sources. In parallel, the Markdown renderer should use DOMPurify to sanitise output, and all third-party CDN scripts should have SRI hashes.

Document prepared as part of the Automation & QA Developer take-home assessment. All findings are based on manual exploratory testing of the public demo at demo.realworld.how.